have changed by the time your initial thread can acquire the lock on `head_mutex`, and not only is the returned tail node no longer the tail, but it's no longer even part of the list. This could then mean that the comparison of `head` to `old_tail` ❷ fails, even if `head` really is the last node. Consequently, when you update `head` ❸ you may end up moving `head` beyond `tail` and off the end of the list, destroying the data structure. In the `correct` implementation from listing 6.6, you keep the call to `get_tail()` inside the lock on `head_mutex`. This ensures that no other threads can change `head`, and `tail` only ever moves further away (as new nodes are added in calls to `push()`), which is perfectly safe. `head` can never pass the value returned from `get_tail()`, so the invariants are upheld.

Once `pop_head()` has removed the node from the queue by updating `head`, the mutex is unlocked, and `try_pop()` can extract the data and delete the node if there was one (and return a `NULL` instance of `std::shared_ptr<>` if not), safe in the knowledge that it's the only thread that can access this node.

Next up, the external interface is a subset of that from listing 6.2, so the same analysis applies: there are no race conditions inherent in the interface.

Exceptions are more interesting. Because you've changed the data allocation patterns, the exceptions can now come from different places. The only operations in `try_pop()` that can throw exceptions are the mutex locks, and the data isn't modified until the locks are acquired. Therefore `try_pop()` is exception-safe. On the other hand, `push()` allocates a new instance of `T` on the heap and a new instance of `node`, either of which might throw an exception. However, both of the newly allocated objects are assigned to smart pointers, so they'll be freed if an exception is thrown. Once the lock is acquired, none of the remaining operations in `push()` can throw an exception, so again you're home and dry and `push()` is exception-safe too.

Because you haven't changed the interface, there are no new external opportunities for deadlock. There are no internal opportunities either; the